PYTHON PROGRAM
C0_4

NAME: SHABANA THASLEEM A.W

COURSE: PYTHON PROGRAMMING

COURSE CODE: CSA0802

ASSIGNMENT: 4

NATURE OF THE ASSIGNMENT: Individual

TOTAL MARKS: 50

SUBMISSION DATE: 12/11/26

PYTHON PROGRAM
C0_4

TASK 1: DIAGNOSE AND REPAIR A POOR SCRIPT

Task 1.1 – Identify at least five design deficiencies (2 Marks)

No.	Design deficiency	What could go wrong?
1	Hard-coded file path	The program may not work on another computer.
2	File is never closed	It may waste system resources.
3	Uses `read()`	The entire file is loaded into memory.
4	Uses bare `except`	All errors are silently ignored.
5	Uses `int()` for amounts	Values like 1250.50 cause an error.
6		

PYTHON PROGRAM
C0_4

Repeated string concatenation

It can reduce performance for large reports.

7

Does not record rejected lines

Incorrect records cannot be identified or corrected.

8

Does not handle empty lines properly

The final empty line may cause an indexing error.

Task 1.2 – Repair the Design (3 Marks)

Corrected Python program

This program uses:

- `with` statement to close the file automatically.
- Line-by-line reading.
- `Decimal` to handle amounts with paise.
- Specific exception handling.
- A list to store rejected records.
- A function that accepts the filename as a parameter.

PYTHON PROGRAM
C0_4

```
from decimal import Decimal, InvalidOperation
import csv

def process_file(filename):
    total = Decimal("0.00")
    report = []
    rejected = []

    try:
        with open(filename, "r", encoding="utf-8", newline="") as file:
            reader = csv.reader(file)

            for line_no, parts in enumerate(reader, start=1):
                if not parts or not any(p.strip() for p in parts):
                    continue

                try:
                    if len(parts) != 3:
                        raise ValueError("Expected 3 fields")
```

Task 1.3 – Design Defence (1 Mark)

The corrected program is safer for a 50,000-line file because it reads one record at a time instead of loading the entire file into memory. The `with` statement automatically closes the file, even when an error occurs. Specific exceptions help identify problems, while rejected records are stored for correction. Using `Decimal` prevents loss of accuracy when processing amounts with paise. Lists and `join()` also avoid repeated string concatenation.

Alternative Concept and Complexity Analysis

Method	Time complexity	Memory complexity
`read()`	O(n)	O(n)
`readlines()`	O(n)	O(n)
Line-by-line iteration	O(n)	O(L)

Where:

- `n` = total number of characters in the file.

- L = maximum length of one line.

Conclusion: Line-by-line reading is more memory-efficient for large files. The report and rejected-record lists still require additional memory proportional to their output.

PYTHON PROGRAM
C0_4

TASK 2: FROM REQUIREMENT TO FILE-PROCESSING DESIGN (8 Marks)

TASK:2.1 Biometric Attendance Files

Each classroom produces a daily text file. Each line contains:

```
register_number,date,status
```

Example:

```
192311001,2026-10-08,P
192311002,2026-10-08,A
192311003,2026-10-08,P
192311004,2026-10-08,L
```

Here:

- P = Present
- A = Absent
- L = Invalid status

Task 2.1 – Model the Data (2 Marks)

Input file format:

CSV format containing three fields:

1. Register number
2. Date
3. Attendance status (P or A)

Data structures:

Use a dictionary to store attendance counts.

```
attendance = {    "192311001": {"present": 20, "absent": 5},
"192311002": {"present": 18, "absent": 7}}
```

Use a list to store damaged records.

```
errors = [    (4, "Invalid attendance status")]
```

Output files:

1. `attendance_report.txt` – Student attendance percentage and students below 75%.
2. `error_log.txt` – Damaged lines with filename and line number.

Attendance percentage formula:

\[\text{Percentage} = \frac{\text{Present}}{\text{Present + Absent}}\times100 \]

Task 2.2 – Design the File Operations (2 Marks)

Operation	Method
Read files	`open(file, "r")`
Write reports	`open(file, "w")`
Write error logs	`open(file, "w")`
Encoding	UTF-8
Read records	Line-by-line iteration
Close files	`with` statement

Reason: The `with` statement automatically closes files and prevents resource leaks.

Task 2.3 – Design the Processing (3 Marks)

Python program for monthly attendance processing

PYTHON PROGRAM
C0_4

```python
import os
import csv

def process_month(folder):
    attendance = {}
    errors = []

    for filename in os.listdir(folder):
        if not filename.endswith(".csv"):
            continue

        path = os.path.join(folder, filename)

        try:
            with open(path, "r", encoding="utf-8") as file:
                reader = csv.reader(file)

                for line_no, row in enumerate(reader, 1):
                    try:
                        if len(row) != 3:
                            raise ValueError("Invalid fields")
```

Task 2.4 – Trace (1 Mark)

```
192311001,2026-10-08,P
192311002,2026-10-08,A
192311003,2026-10-08
192311004,2026-10-08,P
```

TASK 3 – DESIGN THE ERROR HANDLING (9 Marks)

Task 3.1 – Identify the Failures (3 Marks)

The following failures can occur in an attendance-processing program.

No.	Failure	Exception
1	Folder does not exist	`FileNotFoundError`
2	Input file does not exist	`FileNotFoundError`
3	Permission denied	`PermissionError`
4	Wrong file encoding	`UnicodeDecodeError`
5	Damaged or missing fields	`InvalidRecord`
6	Invalid attendance status	`InvalidRecord`
7	Incorrect data conversion	`ValueError`
8	Invalid list or tuple index	`IndexError`
9	File reading or writing failure	`OSError`

Explanation:

Specific exceptions help identify the exact cause of an error. They prevent the program from stopping unexpectedly and make debugging easier.

Task 3.2 – Design a Custom Exception (2 Marks)

```python
class InvalidRecord(Exception):
    """Raised when an attendance record is invalid."""
    pass
```

Task 3.3 – Use the Complete Try Statement (3 Marks)

PYTHON PROGRAM
C0_4

```
# Test data
data = (
    "192311001,2026-10-08,P\n"
    "192311002,2026-10-08,A\n"
    "192311003,2026-10-08\n"
    "192311004,2026-10-08,L\n"
)

with tempfile.NamedTemporaryFile(
    mode="w",
    suffix=".csv",
    encoding="utf-8",
    delete=False
) as tmp:
    tmp.write(data)
    filename = tmp.name

try:
    print(summarise(filename))
finally:
    os.remove(filename)
```

Task 3.4 – Misuse Experiment (1 Mark)

Experiment: Replace specific exception clauses with:

```
except Exception:    pass
```

Introduce a typing error, for example:

```
present += 1
```

Instead of:

```
present += 1
```

Observation:

The program silently ignores the `NameError` caused by the incorrect variable name. It continues executing without displaying the actual problem

PYTHON PROGRAM
C0_4

TASK 4 – FILE AND EXCEPTION FORENSICS (7 Marks)

Task 4.1 – Locate the Defects (2 Marks)

```
def save_returns(books):
    log = open("returns.log", "W")
    for b in books:
        log.write(b)
    log.close()


def process(day_file):
    try:
        books = open(day_file).read().split("\n")
        save_returns(books)
    except FileNotFoundError:
        print("missing")
```

Task 4.2 – Construct Evidence (2 Marks)

```
def save_returns(books):
    with open("returns.log", "w") as log:
        for book in books:
            log.write(book)


# Simulate three days
save_returns(["Book A", "Book B"])
save_returns(["Book C"])
save_returns(["Book D", "Book E"])

with open("returns.log", "r") as file:
    print(file.read())
```

PYTHON PROGRAM
C0_4

Task 4.3 – Repair the Program (2 Marks)

```
import os


def save_returns(books):
    try:
        with open("returns.log", "a", encoding="utf-8") as log:
            for book in books:
                log.write(book.strip() + "\n")

    except OSError as error:
        with open("error.log", "a", encoding="utf-8") as err:
            err.write(f"Write error: {error}\n")
        raise


def process(day_file):
    try:
        with open(day_file, "r", encoding="utf-8") as file:
            books = [line.strip() for line in file if line.strip()]

        save_returns(books)
```

Task 4.4 – Prevent Recurrence (1 Mark)

Two rules for safe file handling:

1. Always use the `with` statement when opening files.
2. Use append mode (`"a"`) when adding new records to an existing log file.

Alternative Concept and Complexity Analysis

If files were opened and closed manually inside `try/finally`, the `finally` block would need to close every successfully opened file.

Method	File handling
Manual open and close	Programmer must close files
`with` statement	Automatically closes files

Time complexity: O(n), where n is the total number of book records.

Space complexity: O(n) for the list of records read into memory.

If records are processed line by line, auxiliary memory can be reduced to O(1), excluding the output buffer.

Conclusion: Using `with`, append mode and error logging makes the library program more reliable and prevents data loss.

PYTHON PROGRAM
C0_4

TASK 5 – THE SAME UTILITY ORGANISED FOUR WAYS (7 Marks)

Task 5.1 – Four Organisations (3 Marks)

(a) Organisation 1 – One Script

```
def read_file(filename):
    records = []

    with open(filename, "r") as file:
        for line in file:
            records.append(line.strip().split(","))

    return records


def calculate_percentage(present, absent):
    total = present + absent

    if total == 0:
        return 0

    return present / total * 100


def write_report(register, percentage):
    with open("report.txt", "a") as file:
```

(b) Organisation 2 – One Module Imported by Main Script

PYTHON PROGRAM
C0_4

```
def calculate_percentage(present, absent):
    total = present + absent

    if total == 0:
        return 0

    return present / total * 100


def read_file(filename):
    with open(filename, "r") as file:
        return file.readlines()


def write_report(register, percentage):
    with open("report.txt", "a") as file:
        file.write(f"{register}: {percentage:.2f}%\n")
```

(c) Organisation 3 – Three Modules in One Folder

```
def read_file(filename):
    records = []

    with open(filename, "r") as file:
        for line in file:
            records.append(line.strip().split(","))

    return records
```

PYTHON PROGRAM
C0_4

Task 5.2 – Change the Requirement (2 Marks)

```
from datetime import datetime, timedelta


def find_three_day_absences(records):
    """
    records format:
    {
        "register_number": {
            "2026-10-08": "A",
            "2026-10-09": "A"
        }
    }
    """

    result = []

    for register, attendance in records.items():

        absent_dates = sorted(
            datetime.strptime(date, "%Y-%m-%d").date()
            for date, status in attendance.items()
```

```
['192311001']
```

Task 5.3 – Compare the Organisations (2 Marks)

Comparison table

Feature	One script	One module	Three modules	Package
Readability	Low	Good	Very good	Very good
Reusability	Low	Good	High	High
Testing	Difficult	Easy	Easy	Easy
Maintenance	Difficult	Good	Very good	Very good
Naming conflicts	More likely	Moderate	Lower with qualified imports	Lower with qualified imports

Difference between import statements

1. `import attendance.stats`

```
import attendance.statsattendance.stats.calculate_percentage(20, 5)
```

Imports the module using its full name.

2. `from attendance import stats`

```
from attendance import statsstats.calculate_percentage(20, 5)
```

Imports the stats module from the attendance package.

3. `from attendance.stats import calculate_percentage`

```
from attendance.stats import
calculate_percentagecalculate_percentage(20, 5)
```

Imports only the required function directly.

Difference:

- First method uses the complete module path.
- Second method imports the module from a package.
- Third method imports a particular function directly.

Direct imports are shorter but may cause naming conflicts if different modules contain functions with identical names.

PYTHON PROGRAM
C0_4

TASK 6 – DESIGN CHALLENGE (13 Marks)

Stage 1 – Interpret the Problem (2 Marks)

1. Inputs

- Five department registration files.
- Student register number.
- Student name.
- Department name.
- E-mail address.
- Selected events.

2. Outputs

1. `master_list.csv` – Clean student registration details.
2. `event_participants.txt` – Event-wise participant lists.
3. `summary_report.txt` – Total registrations and event counts.
4. `error_log.txt` – Damaged records and missing file details.

3. Processing rules

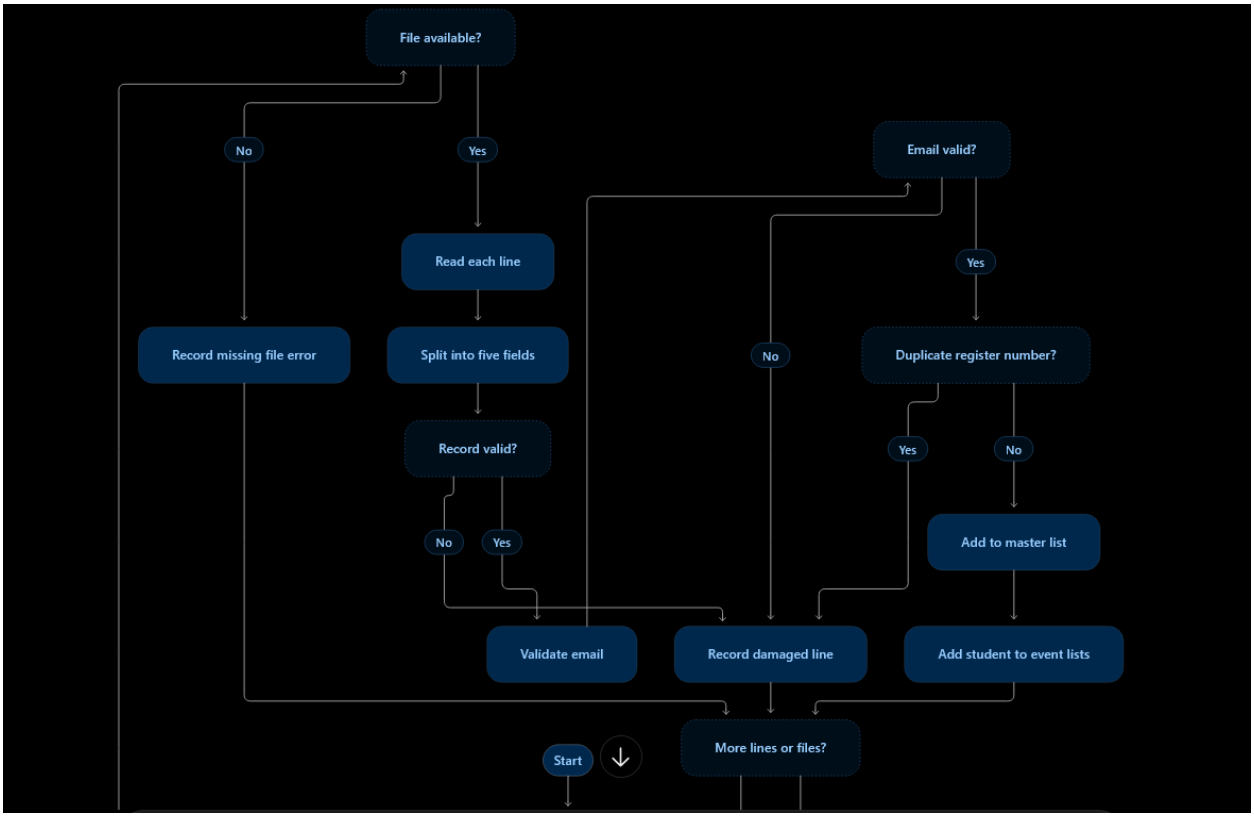
- Each registration must contain exactly five fields.
- Register numbers must not be empty.
- Student names must not be empty.
- E-mail addresses must be valid.
- Duplicate register numbers must be removed.
- Event names must be separated by semicolons.
- Invalid records must be written to the error log.
- A missing department file must not stop the entire program.

Requirements as testable statements

Requirement	Expected result
Five files are provided	All available files are processed
Duplicate register number	Only one registration is retained

Invalid e-mail	Record is rejected
Damaged line	Error is recorded
Missing department file	Error logged, other files processed
Valid registration	Added to master list
Event selected	Student added to event list

Stage 2 – Propose the Processing Flow (2 Marks)



Stage 3 – Identify the Critical Design Decisions (2 Marks)

1. Duplicate detection

Use a Python set to store register numbers that have already been processed.

```
seen_registers = set()if register in seen_registers:
print("Duplicate registration")else:    seen_registers.add(register)
```

2. E-mail validation

Use regular expressions with the `re` module.

```
import redef valid_email(email):    pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"    return re.fullmatch(pattern, email) is not None
```

3. Exception handling

Use specific exceptions:

Exception	Purpose
`FileNotFoundError`	Missing department file
`PermissionError`	File access denied
`UnicodeDecodeError`	Incorrect file encoding
`OSError`	Other file-related errors
`ValueError`	Invalid registration data

Use `try-except` to handle errors and continue processing other department files.

Stage 4 – Design the Package (4 Marks)

ala - IT - bala@gmail.com

summary_report.txt

COLLEGE FEST SUMMARY

Total valid registrations: 4

Event-wise counts:

Dance: 2

Drama: 2

Singing: 2

error_log.txt

department2.txt, Line 2: Duplicate registration

department2.txt, Line 3: Invalid email

department3.txt: File not found

PYTHON PROGRAM
C0_4

department4.txt: File not found

department5.txt: File not found

Alternative Concept and Complexity Analysis

Question: If duplicate registrations were detected by searching a list of all previous registrations instead of using a set, what would happen?

Comparison

Method

Average time complexity

Searching a list for every registration

$O(n^2)$

Using a set for duplicate detection

$O(n)$

Where n is the total number of registrations.

Explanation:

PYTHON PROGRAM
C0_4

If a list is used, the program searches previous registrations every time a new registration arrives. This increases processing time as the number of registrations grows.

Using a set provides average O(1) lookup, making duplicate detection faster.

Space complexity:

Master list: O(n)

Register number set: O(n)

Event participant dictionary: O(n + e), where e is the total number of event selections.

Error log: O(r), where r is the number of rejected records.

Conclusion: Using a set is more efficient for detecting duplicate registrations. It reduces processing time and makes the registration system suitable for large numbers of students.